

1. Validate URL, scope, protocol, redirect behavior, and scan policy.
2. Discover endpoints using crawling, browser navigation, known API paths, and OpenAPI/Swagger where exposed.
3. Identify authentication mechanisms and establish permitted test identities when supplied.
4. Build an application model containing endpoints, parameters, methods, cookies/tokens, and observed responses.
5. Execute focused tests for authentication, authorization/BOLA, SSRF, basic injection, and security misconfiguration.
6. Collect requests, responses, status codes, timing, headers, authentication identity, and controlled callback evidence.
7. Normalize confirmed and suspected findings with severity and confidence.
8. Calculate risk and produce a runtime-only report.

Example runtime finding

Finding: BOLA / Broken Object Level Authorization
Endpoint: GET /api/users/{id}

User A -> /api/users/102 -> 200
User B -> owns object 102

Conclusion:
User A can access an object belonging to User B.

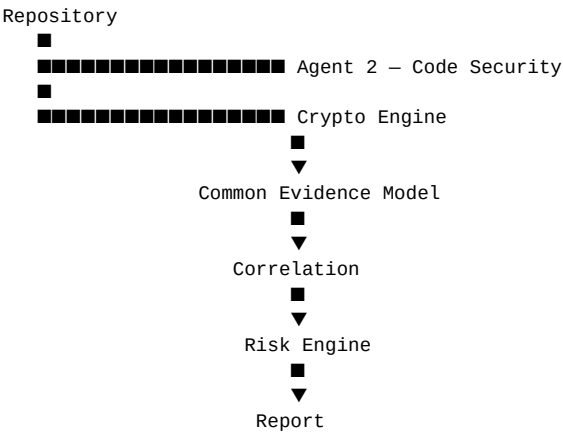
Available evidence:
HTTP request/response + authenticated identity + object ownership context

Tools used by Agent 1

Tool	Purpose
Python	Orchestration, state management, test logic
httpx	Controlled HTTP/API requests
Playwright	Browser-based authentication and complex flows
OpenAPI parser	Parse Swagger/OpenAPI specifications
Crawler/discovery tool	Endpoint and application-surface discovery
Nuclei	Reusable template-based checks where appropriate

6. Mode 2 — Detailed Working: Repository Only

When only a repository is supplied, AEGIS performs source-level and cryptographic analysis. Runtime testing is disabled because there is no deployed target.



Agent 2 workflow

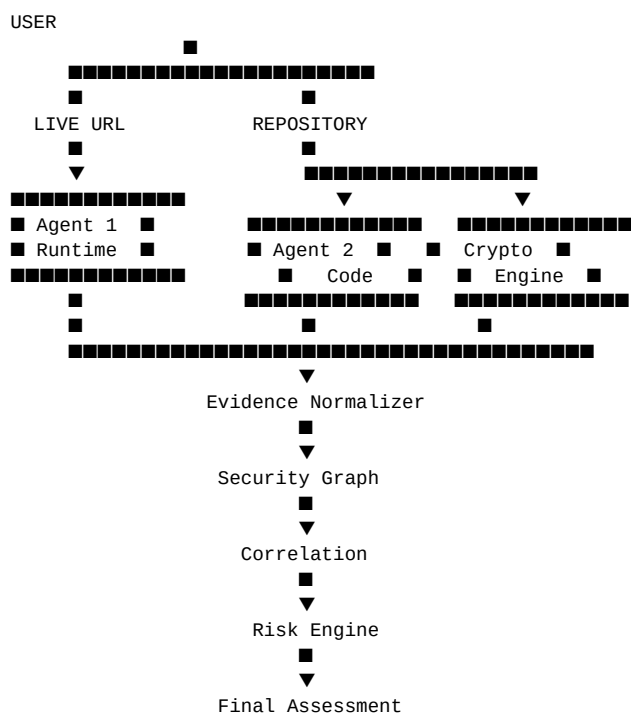
1. Acquire and validate the repository or local source tree.
2. Inventory languages, frameworks, package manifests, entry points, API routes, authentication code, database access, configuration, Docker/IaC, and tests.
3. Build a multi-language code model using AST-aware parsing.
4. Run SAST rules and identify source-level vulnerability patterns.
5. Analyze dependencies and known vulnerabilities.
6. Detect secrets and sensitive configuration.
7. Analyze infrastructure-as-code and deployment configuration where supported.
8. Normalize findings and attach file/function/line/data-flow evidence.
9. Pass findings to the shared risk/correlation layer.

Agent 2 tools

Tool	Purpose
Python + FastAPI	Agent orchestration and service layer
Semgrep / OpenGrep	Static security rules and source pattern detection
Tree-sitter	AST/code parsing and call-site context
OSV-Scanner	Open-source dependency vulnerability analysis
Gitleaks	Secret and credential detection
Checkov	Infrastructure-as-code security checks
Git provider/local filesystem	Repository acquisition

7. Mode 3 — Detailed Working: URL + Repository

This is the complete AEGIS assessment and the primary differentiator. Agent 1 establishes runtime facts; Agent 2 identifies source-level causes; the Crypto Engine identifies cryptographic assets and risks. The shared correlation layer connects these evidence streams.



Example of correlation

Agent 1:
GET /api/users/{id} is exploitable as BOLA.

Agent 2:
users.py:get_user() retrieves the object without an object-level authorization check.

AEGIS:
Same endpoint + same function + matching behavior
-> one high-confidence correlated finding.

Result:
Runtime evidence proves impact.
Source evidence identifies root cause.

8. Agent 1 — Runtime Security Architecture

Agent 1

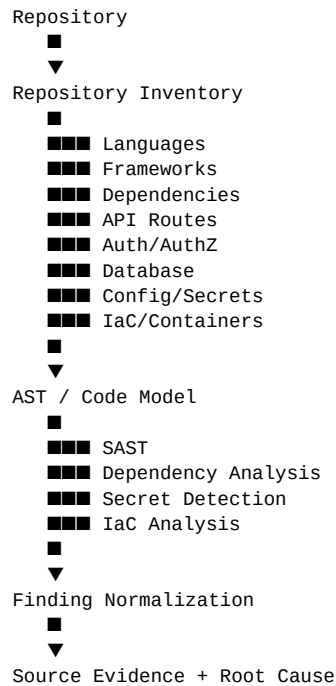
- Orchestrator
- Scope & Safety Controller
- Discovery
 - ■■■ Crawler
 - ■■■ OpenAPI Discovery
 - ■■■ Browser Discovery
- Authentication Analyzer
- Authorization / BOLA Tester
- SSRF Tester
- Injection Tester
- Misconfiguration Checks
- Evidence Collector
- Finding Normalizer
- Reasoning / Explanation Layer

The orchestrator manages state and sequencing. Deterministic testing modules perform actual requests and checks. The evidence collector records the facts needed to reproduce or validate a finding.

MVP vulnerability coverage

Area	Example	Primary evidence
Authentication	Weak login/session behavior	Request/response + session state
Authorization/BOLA	Cross-user object access	Two identities + object ownership + response
SSRF	Server follows attacker-controlled URL	Controlled callback/canary + request evidence
Injection	Basic SQL/command/template injection indicators	Controlled payload + response/behavior
Misconfiguration	Unsafe headers/configuration/exposed services	Observed headers/configuration behavior

9. Agent 2 — Code Security Architecture



Agent 2 is strongest when it can connect a finding to a precise file, function, line, and data flow. The LLM receives structured evidence rather than raw repository text wherever possible.

Example

```
@app.get("/api/users/{id}")
def get_user(id):
    return db.get_user(id)
```

Code Agent interpretation:

- Endpoint retrieves a user-controlled object identifier.
- No visible object-level authorization guard exists.
- Candidate root cause for BOLA.
- Confidence increases if runtime Agent 1 independently confirms cross-user access.

10. Crypto Engine — ECDAT Architecture

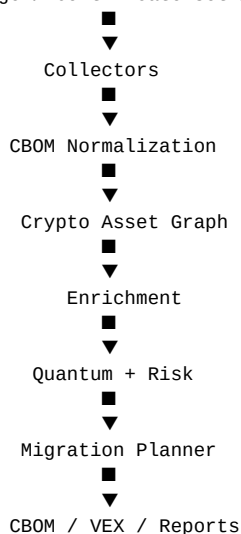
The Crypto Engine is a specialized repository-level cryptographic discovery and decision engine. The plan is based on the ECDAT concept: Enterprise Cryptographic Discovery & Analysis Tool. Its purpose is not merely to flag weak algorithms; it builds a normalized Cryptography Bill of Materials (CBOM), enriches assets with context, evaluates quantum and classical risk, estimates migration difficulty, and recommends migration priorities.

ECDAT principle

Discover → Normalize → Enrich → Assess → Prioritize → Recommend → Export.

The Crypto Engine is activated whenever a repository is supplied. It can operate independently in repository-only mode or alongside Agent 1 in the full URL + repository mode.

Repository / Image / Certificate set / Protocol evidence



11. Crypto Engine — Collectors and Tools

Collectors are designed as plugins so the engine can add new evidence surfaces without changing the core risk model.

Collector	What it finds	Primary tools	Evidence confidence
Source	Algorithms, modes, padding, key sizes, crypto APIs, hardcoded keys	OpenGrep/Semgrep crypto rules + Tree-sitter	High for typed API matches; lower for strings/comments
Dependency	Crypto libraries and transitive packages	Manifest parsers + Syft + library signature registry	High
Certificate & Key	Algorithm, key size, signature, validity, chain, PEM/DER/PKCS#12/JKS material	Python cryptography + pyjks	High for literal certificate fields
Binary	Crypto constants, symbols, library/version indicators	YARA/findcrypt rules + LIEF + strings + cve-bin-tool signatures	Medium / best-effort
Container	Crypto artifacts inside OCI/TAR image layers and security configuration	Layer unpacking + cbomkit-theia + re-run collectors	Inherited from collectors
Protocol	Negotiated TLS/SSH/IKE algorithms, suites, KEX, certificates; optional PCAP evidence	CryptoLyzer + tshark/pyshark	High for observed protocol fields

Important: binary detection is explicitly labelled best-effort. A detected constant or version string may prove presence but does not necessarily prove that the primitive is used in a security-sensitive path.

12. Crypto Engine — Detailed Workflow

Stage 1 — Discovery

- Scan source code for cryptographic APIs and algorithm identifiers.
- Inspect package manifests and transitive dependencies.
- Inspect certificates, keystores, and cryptographic material where authorized.
- Inspect binaries and container layers.
- Optionally inspect live protocol negotiations and PCAP handshakes.

Stage 2 — Normalization

- Map different detector names to normalized cryptography definitions.
- Deduplicate the same algorithm discovered by multiple collectors.
- Retain all evidence entries instead of discarding duplicate observations.
- Build provides/dependsOn relationships between applications, libraries, images, and crypto assets.

Stage 3 — Context enrichment

- Classify use as production/test/example/vendored/comment where possible.
- Determine security-sensitive versus non-security use.
- Determine exposure: internal/external, in-transit/at-rest, confidentiality/signing.
- Infer data sensitivity and lifetime, while showing assumptions to the user.
- Calculate crypto-agility characteristics from call-site evidence.

Stage 4 — Risk assessment

- Classify quantum impact using Shor/Grover/legacy/safe categories.
- Apply HNDL reasoning only where confidentiality primitives are actually relevant.
- Evaluate migration timing with Mosca-style $X + Y > Z$ reasoning.
- Overlay business criticality and applicable policy/compliance milestones.
- Assign EXPOSED, ACT NOW, MONITOR, or SAFE tiers.

13. Crypto Engine — Risk & Decision Model

The Crypto Engine is intentionally a decision engine over the CBOM rather than a red/green scanner.

Quantum classification

Class	Examples	Meaning
Shor-broken	RSA, DH, ECDH, ECDSA, EdDSA, X25519	Public-key primitives vulnerable to sufficiently capable quantum attacks
Grover-weakened	AES-128, selected hash uses	Quantum search changes security margin; context matters
Legacy-broken	MD5, SHA-1, DES, 3DES, RC4, undersized keys	Classically weak regardless of quantum risk
Safe / PQC	AES-256, SHA-384/512, SHA-3, ChaCha20-Poly1305, ML-KEM, ML-DSA, SLH-DSA	Current quantum-safe or appropriately strong choices, subject to usage context

Mosca-style reasoning

X = data / protection lifetime
Y = migration time
Z = assumed quantum-risk deadline

If $X + Y > Z$:
migration must be accelerated

Z is intentionally treated as a user-adjustable assumption rather than a claimed date for when a cryptographically relevant quantum computer will exist.

HNDL scope

Harvest-now-decrypt-later primarily affects confidentiality: data encrypted or key-exchanged today may be captured for future decryption. Signature weaknesses have a different timing model and should be scored against relevant migration deadlines rather than mechanically applying the same formula.

14. Crypto Engine — Tiers and Crypto Agility

Tier	Definition	Action
ALREADY EXPOSED	Confidentiality asset can be captured now and $X + Y$ exceeds Z	Consider immediate migration, re-keying, retention reduction, or explicit acceptance
ACT NOW	Risk is near/over the migration boundary or a hard policy deadline applies	Prioritize migration
MONITOR	Quantum-vulnerable but short-lived, low criticality, or signing-only context	Track and prepare
SAFE	Quantum-safe or demonstrably non-security use	No migration required; continue governance

Crypto-agility is not represented as one unexplained magic number. The engine records dimensions such as:

- Algorithm coupling — whether the algorithm is hardcoded.
- Provider/library coupling — direct dependency versus abstraction.
- Parameter coupling — hardcoded key sizes, modes, curves, etc.
- Decoupling mechanism — configuration, environment, runtime negotiation.
- Spread — number of call sites/files affected.
- Ownership — first-party, vendored, or opaque binary.
- Runtime availability — whether suitable replacement primitives are already available in the environment.

This enables the dashboard to show risk × migration difficulty rather than severity alone.

15. Crypto Engine — CBOM as the Contract

All cryptographic evidence converges into a normalized CBOM. The CBOM becomes the single source of truth for downstream risk, migration planning, exports, and governance.

```
CryptoAsset
  ■■■ bom_ref
  ■■■ asset_type
  ■■■ name / normalized definition
  ■■■ primitive
  ■■■ parameter_set
  ■■■ mode
  ■■■ padding
  ■■■ crypto_functions[]
  ■■■ classical_security_level
  ■■■ quantum_security_level
  ■■■ evidence[]
    ■ ■■■ collector
    ■ ■■■ location
    ■ ■■■ confidence
    ■ ■■■ snippet / observation
  ■■■ component
  ■■■ provided_by
  ■■■ exposure
  ■■■ usage_context
```

Example reconciliation

Source collector:
RSA key exchange observed in application configuration.

Container collector:
Same RSA configuration observed in image layer.

Protocol collector:
RSA-based TLS behavior observed during live handshake.

CBOM:
ONE crypto asset
+ THREE evidence records
+ relationships to application, image and endpoint

Planned outputs include CycloneDX CBOM, VEX-style status, SARIF, PDF/HTML reports, and CSV. The CBOM should be validated and cryptographically signed in the mature implementation.

16. Common AEGIS Intelligence Layer

The common layer is what turns individual scanners into AEGIS.

Evidence Normalizer

- Converts runtime, code, and crypto observations into a common schema.
- Deduplicates related findings while preserving all evidence.
- Assigns source, confidence, location, affected asset, and relationships.

Security Graph

Application

- Endpoint Function Finding
- Endpoint Data Asset
- Function Library
- Library Crypto Asset
- Crypto Asset Certificate/Key
- Finding Finding / Attack Path

Correlation examples

- Runtime BOLA + source missing authorization → confirmed finding with root cause.
- Runtime SSRF + source server-side URL fetch → confirmed SSRF with source location.
- Source RSA configuration + container configuration + live TLS handshake → one cryptographic asset with multiple evidence sources.
- Hardcoded credential + reachable endpoint + sensitive data path → higher attack-path priority.

17. Risk Engine and LLM Reasoning

Risk Engine

The Risk Engine combines technical severity with evidence confidence, exploitability, exposure, asset criticality, sensitive-data relevance, and graph connectivity. The output is a prioritized remediation queue rather than an unordered list.

LLM Reasoning Layer

The LLM is used for interpretation and communication after deterministic evidence exists. It can explain findings, correlate evidence, summarize attack paths, reason about likely root causes, generate remediation guidance, and produce developer-friendly reports.

It should not be allowed to invent test results. For every claim, the reasoning layer should reference structured evidence such as an HTTP observation, source file/line, crypto API, certificate field, or protocol field.

Bad:

URL → LLM → "20 vulnerabilities"

AEGIS:

URL → security tools → evidence

↓

normalized findings

↓

security graph

↓

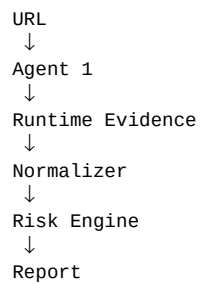
LLM

↓

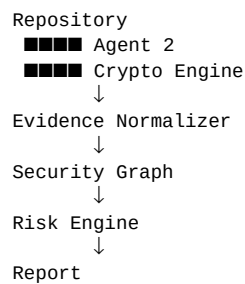
explanation / correlation /
remediation / reporting

18. Architecture Summary by Case

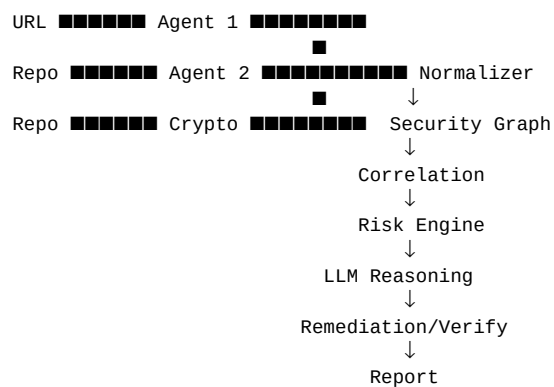
Case 1 — URL only



Case 2 — Repository only



Case 3 — URL + Repository



19. Final Report and User Experience

AEGIS presents a single assessment even when multiple engines contributed evidence. The report distinguishes what was actually tested from what could not be tested because an input was unavailable.

URL-only example

```
Coverage
✓ Runtime analysis
✗ Source-code analysis – repository not provided
✗ Crypto analysis – repository not provided

Findings
HIGH  BOLA – GET /api/users/{id}
HIGH  SSRF – POST /api/fetch-url
MED   Security header weakness
```

Full assessment example

```
Coverage
✓ Runtime
✓ Source code
✓ Cryptography
✓ Correlation
✓ Risk analysis

Top finding
BOLA – Confirmed

Runtime:
User A accessed User B's resource.

Source:
users.py:get_user() lacks object-level authorization.

Impact:
Unauthorized access to sensitive user data.

Recommendation:
Implement object-level authorization before returning the object.
Re-test with two authorized identities after remediation.
```

Each finding should contain: title, OWASP/crypto category, severity, confidence, affected asset, evidence, source location, root cause, impact, remediation, and verification state.

20. Security, Safety and Trust Controls

- Authorized-scope requirement for runtime testing.
- Rate limits and controlled payloads to reduce unintended impact.
- Safe SSRF validation through controlled callback/canary infrastructure rather than arbitrary internal exploitation.
- No claim of source-level root cause without source evidence.
- No claim that a binary constant proves active cryptographic use.
- Confidence levels on every finding.
- LLM reasoning constrained by structured evidence.
- Patch suggestions are recommendations, not blind automatic modifications.
- Offline mode for sensitive environments.
- Secrets discovered during scanning should be protected in logs and reports, with redaction where appropriate.

AEGIS should distinguish confirmed, probable, and potential findings. Similarly, attack paths should be labelled confirmed or hypothetical depending on the evidence available.

21. Proposed Technology Stack

Layer	Technology
Backend / orchestration	Python 3.12, FastAPI
Runtime HTTP	httpx
Browser automation	Playwright
Runtime templates	Nuclei where appropriate
Code SAST	Semgrep / OpenGrep
AST	Tree-sitter
Dependencies	OSV-Scanner, manifest parsers, Syft
Secrets	Gitleaks
IaC	Checkov
Crypto parsing	Python cryptography, pyjks
Binary analysis	YARA/findcrypt, LIEF, strings, cve-bin-tool signatures
Container analysis	OCI/TAR layer unpacking, cbomkit-theia integration
Protocol analysis	CryptoLyzer, tshark/pyshark
CBOM	CycloneDX tooling / cyclonedx-python-lib
Database	PostgreSQL
Async workers	Redis + Celery/RQ or equivalent
Frontend	React + TypeScript + Vite
Optional local AI	Ollama / local LLM
Interfaces	CLI, REST API, dashboard, CI/CD

22. Implementation Plan

Phase 1 — AEGIS foundation

- Build orchestrator and input router.
- Implement Agent 1 MVP: discovery, authentication, BOLA, SSRF, basic injection, misconfiguration.
- Implement Agent 2 MVP: repository inventory, SAST, dependencies, secrets.
- Define common finding/evidence schema.

Phase 2 — Crypto Engine foundation

- Implement source crypto collector.
- Add dependency and certificate/key collectors.
- Normalize crypto assets and emit initial CBOM.
- Implement algorithm knowledge base and basic risk classification.

Phase 3 — Full correlation

- Implement Security Graph.
- Correlate runtime endpoint/function with source findings.
- Add container, binary and live protocol collectors.
- Add crypto-agility and risk × agility visualization.

Phase 4 — Decision layer

- Implement HNDL-aware Mosca-style model.
- Add migration tiers and user-adjustable Z assumption.
- Add PQC/hybrid recommendation mappings.
- Add migration optimizer, VEX, CI gate, signed CBOM and richer reports.

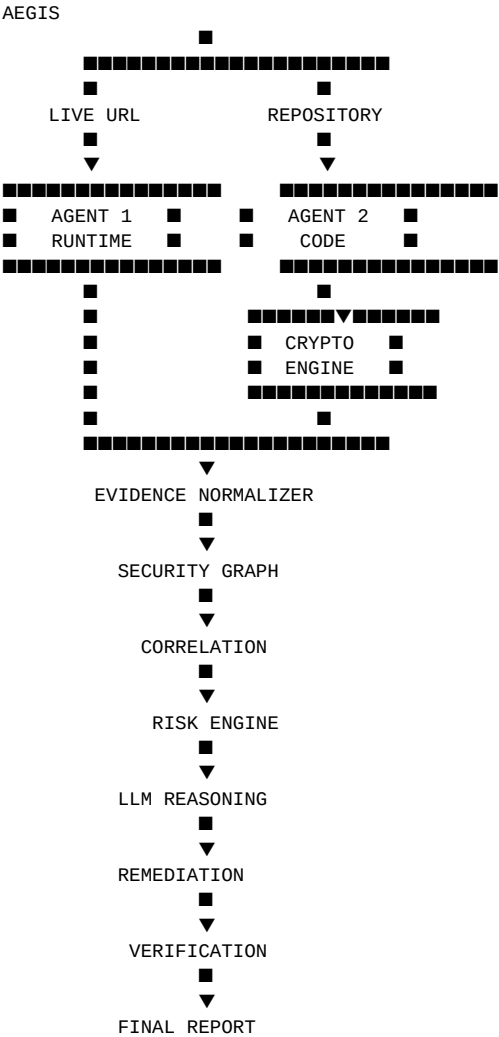
23. Recommended Hackathon Demonstration

The strongest demonstration uses the same intentionally vulnerable application for both the live URL and source repository.

1. Enter the deployed URL and start a URL-only scan.
2. Show Agent 1 discovering an exploitable BOLA and SSRF with concrete evidence.
3. Add the GitHub repository and rerun the complete assessment.
4. Show Agent 2 mapping the BOLA to the responsible source function.
5. Show the Crypto Engine finding a weak cryptographic implementation or hardcoded key.
6. Open the Security Graph and demonstrate the relationship between endpoint, function, finding, and crypto asset.
7. Show risk prioritization and remediation recommendations.
8. Fix one vulnerability and run a focused re-test to demonstrate verification.
9. Export the final report/CBOM.

Judge-facing message: “Agent 1 tells us what the deployed application is doing. Agent 2 tells us why the vulnerability exists. The Crypto Engine tells us where cryptography is weak and how difficult it is to migrate. AEGIS connects the evidence and tells the engineering team what to fix first.”

24. Final Architecture — Frozen Product Definition



Final product statement

AEGIS adapts its security assessment to the evidence available. A URL enables runtime security testing. A repository enables source-code and cryptographic analysis. Providing both enables AEGIS to correlate runtime behavior with source-level root causes and cryptographic risk, producing one evidence-backed security assessment.

This architecture keeps the three engines modular, makes each scan mode independently useful, and reserves the strongest value proposition — cross-layer correlation and decision support — for the complete URL + repository assessment.

Appendix: ECDAT implementation notes

- Keep cryptographic knowledge in auditable YAML/data files rather than hardcoding policy into the engine.
- Treat algorithm classification, deadlines, PQC mappings, data lifetimes, and framework overlays as versioned knowledge.
- Keep collector plugins hot-swappable.
- Preserve raw evidence so findings can be reproduced and audited.

- Measure precision, recall, F1, CBOM validity, completeness, scan time, and reconciliation accuracy on a labelled crypto test corpus.
- Do not overclaim quantum timelines; expose assumptions and let the user adjust the risk horizon.